



S1. Baseline Models

"We need to check if the problem is solvable in the first place"

Contents:

[1. What is a Baseline?](#)

[2. A Baseline Model](#)

[3. Beating the Baseline](#)

[4. Baseline for Time Series](#)

[Summary](#)

[References](#)

When learning Machine Learning or Deep Learning, you will eventually encounter a term that appears surprisingly often: **the baseline**. We have briefly mentioned it before when discussing techniques for improving model performance. Before we can improve a model, however, we need something to improve from. That starting point is what we call a baseline.

At first glance, baselines may seem uninteresting. After all, why spend time building a weak model when our goal is to create a strong one? But this is exactly where many beginners make a mistake. Before we worry about advanced architectures, hyperparameter tuning, feature engineering, or state-of-the-art Deep Learning models, we need to answer a much simpler question: **Is this problem even solvable?**

A baseline helps us answer that question. It provides a reference point, a minimum level of performance that any serious solution should be able to beat. Without a baseline, it becomes difficult to know whether our sophisticated model is genuinely learning something useful or simply producing numbers that look impressive.

In many ways, a baseline acts as a reality check. It tells us whether the data contains meaningful signals, whether our evaluation process is working

correctly, and whether investing more effort into the problem is worthwhile. Before building the most advanced model we can imagine, we first build the simplest model we can justify.

And surprisingly often, that simple model teaches us far more than we expect.

1. What is a Baseline?

One interesting thing about baselines is that there is no single universal definition. The exact form of a baseline often depends on the type of problem we are trying to solve.

- For a time-series forecasting problem, a baseline might simply predict that tomorrow will be the same as today. If a stock price is \$100 today, the baseline predicts \$100 tomorrow. No complex mathematics. No Machine Learning. Just a simple assumption that the future resembles the recent past.
- For a text classification problem, a baseline might classify a document using only the first sentence, the first few words, or even the most common class in the dataset. If 70% of all emails are not spam, a baseline could simply predict "not spam" every single time.
- For an image classification problem, a baseline might be a very small neural network, a logistic regression model, or even a random guess weighted by class frequencies.

Although these examples look very different, they all share the same philosophy. A baseline is not designed to be the best solution. It is designed to be the **simplest reasonable solution**.

In other words, a baseline asks: ***"If we put in the minimum amount of effort, how well can we already solve this problem?"***

This question is far more important than it appears.

- Suppose we spend weeks building a sophisticated Deep Learning model and achieve 85% accuracy. At first, that sounds impressive.
- But if a trivial baseline already achieves 83% accuracy, then our expensive model may not be providing much value.
- On the other hand, imagine a baseline achieves only 55% accuracy, while our final model reaches 85%. Suddenly, that improvement becomes much

more meaningful.

This is why baselines are not merely simple models. They are measuring tools. They establish a starting point that allows us to evaluate every future improvement objectively.

We can think of a baseline as the Machine Learning equivalent of a control group in an experiment. Before claiming that a new technique works, we first need something to compare it against.

And perhaps the most important lesson is this: A baseline is not built to win. It is built to answer a question. **Can this problem be solved better than a simple guess?**

If the answer is no, then no amount of advanced Machine Learning is likely to save us. If the answer is yes, then we know the problem contains learnable patterns, and we have a solid foundation upon which to build better models.

2. A Baseline Model

So we will define a **Baseline Model** as follows:

A baseline model is the easiest reasonable model possible for solving a given problem.

Notice that we intentionally use the word *reasonable*. A baseline is not simply a random guess. It should represent the minimum level of effort that still makes sense for the problem domain.

For example, suppose we are trying to predict tomorrow's temperature. A completely random prediction would not be a useful baseline because nobody would seriously use such a strategy. However, predicting that tomorrow's temperature will be the same as today's temperature is a simple and reasonable approach. It requires almost no computation, yet it still captures an important pattern that often exists in weather data.

Similarly, if we are building a spam detection system and 90% of emails are legitimate, one baseline could simply predict "not spam" for every email. This sounds almost silly, but it immediately gives us a reference point. Any Machine Learning model that cannot outperform this simple strategy is probably not learning anything useful.

The key idea is that a baseline should be easy to understand, easy to implement, and easy to evaluate. We should be able to build it quickly and obtain a performance score that serves as a benchmark for all future models.

Think about it like climbing a mountain. Before we can measure how far we have climbed, we need to know where the ground is. The baseline provides that ground level. Without it, numbers such as 80% accuracy or a Mean Absolute Error of 5 become difficult to interpret because we have nothing to compare them against.

This is why experienced Machine Learning practitioners rarely start with complex models. Instead, they often begin with a baseline and ask a very simple question: *"Can we beat this?"*

- If the answer is no, then either the problem is extremely difficult, the data is poor, or our advanced model is not working correctly.
- If the answer is yes, then we have evidence that our additional complexity is actually providing value.

In practice, a baseline model serves three important purposes:

- **It verifies that the problem is solvable.**
- **It provides a benchmark for comparison.**
- **It prevents us from overestimating the value of complex models.**

Many beginners assume that Machine Learning is a competition to build the most sophisticated model possible. In reality, Machine Learning is a process of demonstrating improvement. And improvement can only be measured when we have a starting point. **That starting point is the baseline model.**

3. Beating the Baseline

Now that we understand what a baseline model is and why we build one, we can move on to an equally important question: **What does it actually mean to beat the baseline?**

At first, the answer seems obvious. If our Machine Learning model performs better than the baseline, then we have beaten it. But there is a deeper idea here.

Remember why we created the baseline in the first place. The baseline represents the simplest reasonable solution to the problem. If our sophisticated

Machine Learning model cannot outperform that simple solution, then something is wrong. Either the problem is harder than we thought, the data contains very little useful information, or our model is simply not learning meaningful patterns.

This is why “beating the baseline” is often considered the first real milestone of a Machine Learning project.

- Let’s look at a simple example. Suppose we want to determine whether a student passes or fails a course based on their score.
 - The simplest baseline could be: **$Score \geq 5 \rightarrow Pass$; $Score < 5 \rightarrow Fail$.**
 - This baseline requires almost no intelligence. It is simply applying a predefined rule.

Now imagine we build a Machine Learning model using attendance records, assignment scores, participation, and exam performance. If the model cannot perform better than this simple rule, then we must ask ourselves whether the additional complexity is justified.

The goal is not to build a complicated model. The goal is to build a model that provides more value than the simplest available solution.

- Let’s consider a more practical Machine Learning example. Suppose we are training a time-series forecasting model to predict tomorrow’s sales.
 - A common baseline is called the **naive forecast**: **$Tomorrow's\ sales = Today's\ sales$**
 - Imagine the baseline achieves a Mean Absolute Error (MAE) of 12 units.
 - After several hours of feature engineering and model training, our advanced neural network achieves an MAE of 15 units.
- Although neural networks sound impressive, the result tells a different story. The baseline is actually better. In practice, we would prefer the simple one-line prediction because it is both easier and more accurate.
 - Now imagine the neural network achieves an MAE of 7 units. In this case, we can confidently say that the model has learned useful patterns beyond what the baseline could capture.

This is what beating the baseline means: ***Demonstrating that the model learns information that the simplest solution cannot.***

There is another important detail that beginners often overlook. **A baseline is not fixed.** The same problem can have multiple baselines.

- Suppose we are evaluating student performance. As mentioned earlier, the most common baseline might be: **Score $\geq 5 \rightarrow$ Pass; Score $< 5 \rightarrow$ Fail.**
- But this is not the only possibility. Perhaps most students score above 8, making the passing threshold uninformative. In that case, we might define a stronger baseline using the class average:
 - Score above average $\rightarrow$ Good performance
 - Score below average $\rightarrow$ Poor performance

Or we might use a z-score to identify students who significantly outperform their peers:

$$z = \frac{x - \mu}{\sigma}$$

where: x is the student's score; μ is the class average; σ is the standard deviation

Now our goal is no longer determining who passes. Instead, we are identifying exceptional students. Notice how the problem remains the same—student evaluation—but the baseline changes depending on what we care about.

In fact, experienced data scientists often build multiple baselines:

- A naive baseline
- A statistical baseline
- A simple Machine Learning baseline
- A more advanced baseline

Each one raises the bar a little higher. And this leads us to an important realization. Beating a weak baseline is necessary, but it is rarely sufficient.

If our model only outperforms a trivial baseline by a tiny amount, we may still question whether the added complexity is worthwhile. The strongest models are not those that merely beat the easiest baseline. They consistently outperform increasingly difficult baselines while maintaining good generalization.

- So whenever you evaluate a Machine Learning model, don't immediately ask: *"How accurate is it?"*

- Instead, ask: *"Accurate compared to what?"*

Because without a baseline, performance numbers have very little meaning. And in many Machine Learning projects, the first victory is not achieving state-of-the-art results. It is simply proving that we can beat the baseline.

4. Baseline for Time Series

Now that we understand what a baseline model is, let us build one ourselves.

In this section, we will compare a simple baseline against a Recurrent Neural Network (RNN) on a time-series forecasting task. The goal is not to achieve the highest possible accuracy. Instead, the goal is to answer a simple question:

Can our RNN learn patterns that a baseline cannot? To keep the example simple, we will generate a synthetic time series using NumPy.

- **Step 1: Generate Time Series Data**

```
import numpy as np
import matplotlib.pyplot as plt

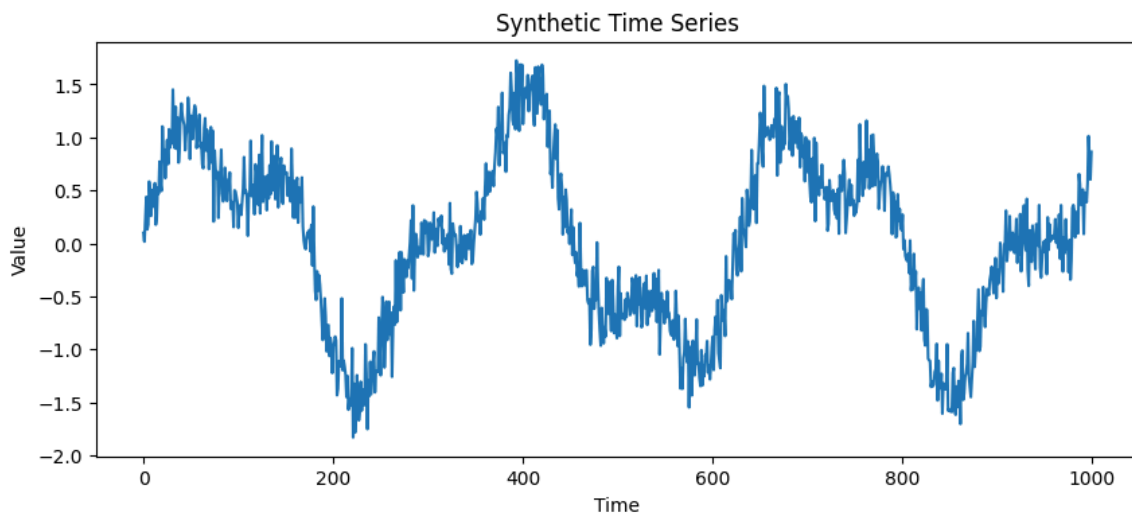
# Reproducibility
np.random.seed(42)

# Generate time points
t = np.arange(1000)

# Create a noisy sine wave
series = (np.sin(0.02 * t) + 0.5 * np.sin(0.05 * t)
          + np.random.normal(0, 0.2, len(t)))

plt.figure(figsize=(10, 4))
plt.plot(series)
plt.title("Synthetic Time Series")
plt.xlabel("Time")
plt.ylabel("Value")
plt.show()
```

The generated data contains a repeating pattern mixed with random noise. This makes it suitable for demonstrating forecasting techniques.



- **Step 2: Create Sliding Windows**

For time-series forecasting, we typically use previous observations to predict the next observation. Suppose we use the previous 20 values to predict the next value.


```

window_size = 20

X = []
y = []

for i in range(len(series) - window_size):
    X.append(series[i:i+window_size])
    y.append(series[i+window_size])

X = np.array(X)
y = np.array(y)

print(X.shape)
print(y.shape)

```

```

(980, 20)
(980,)

```

Example:

Input Window:

```
[10, 12, 13, 15, 16]
```

Target:

```
18
```

The model learns to use historical observations to estimate the next value.

- **Step 3: Train-Test Split**

```

split = int(len(X) * 0.8)

X_train = X[:split]
X_test = X[split:]

y_train = y[:split]
y_test = y[split:]

```

RNNs require a three-dimensional input:

```
X_train_rnn = X_train.reshape(X_train.shape[0],
                              X_train.shape[1],
                              1)

X_test_rnn = X_test.reshape(X_test.shape[0],
                            X_test.shape[1],
                            1)
```

- **Step 4: Build an RNN**

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([SimpleRNN(64, activation='tanh',
                              input_shape=(window_size,
                              1)),
                    Dense(1)])

model.compile(optimizer='adam', loss='mse',
              metrics=['mae'])

history = model.fit(X_train_rnn, y_train,
                    validation_split=0.2,
                    epochs=100, batch_size=32,
                    verbose=1)
```

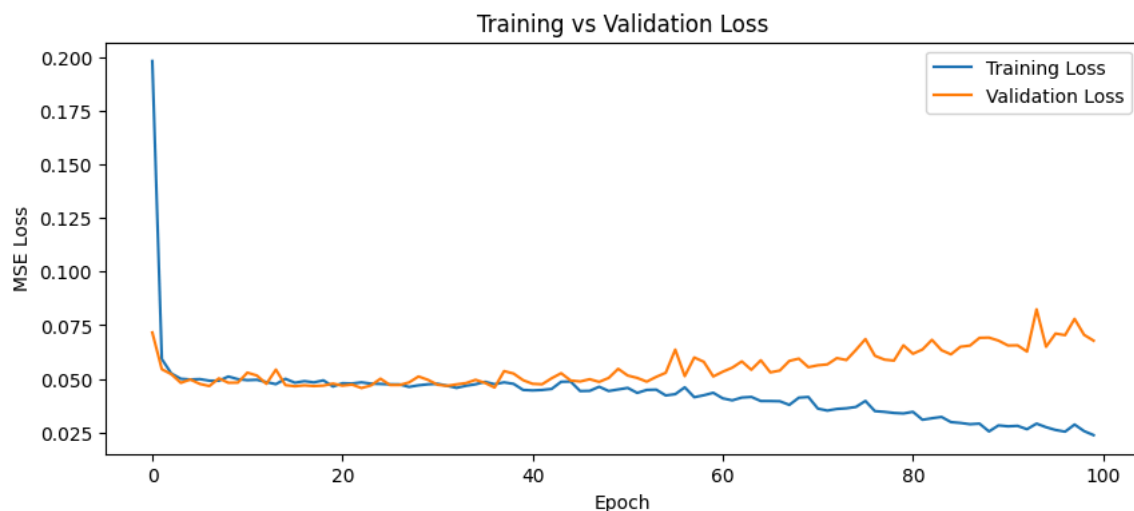
- **Step 5: Analyze Training Curves**

```
plt.figure(figsize=(10,4))

plt.plot(history.history['loss'],
         label='Training Loss')
```

```
plt.plot(history.history['val_loss'],
         label='Validation Loss')

plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()
```



When analyzing neural networks, beginners often focus entirely on accuracy. For forecasting problems, however, the loss curve is usually much more informative.

A common pattern looks like this:

- Training loss continues decreasing.
- Validation loss decreases initially.
- Validation loss starts increasing after a certain epoch.

This is the point where **overfitting** begins. The model continues memorizing training data while becoming worse at generalizing to unseen data.

The epoch corresponding to the minimum validation loss is usually the best model checkpoint.

- **Step 6: Create the Baseline**

For time-series forecasting, one of the most common baselines is the **naive forecast**:

Predict the next value using the most recent value. Mathematically: $\hat{y}_t = y_{t-1}$

Implementation:

```
baseline_pred = X_test[:, -1]
```

No training. No neural network. No optimization. Just copy the last observed value.

- **Step 7: Compare Performance**

```
from sklearn.metrics import mean_absolute_error

rnn_pred = model.predict(X_test_rnn).flatten()
baseline_mae = mean_absolute_error(y_test, baseline_pred)

rnn_mae = mean_absolute_error(y_test, rnn_pred)

print("Baseline MAE:", baseline_mae)
print("RNN MAE:", rnn_mae)
```

Baseline MAE: 0.21358277580775226

RNN MAE: 0.21092978583405092

The RNN produces smaller prediction errors, meaning it successfully learned patterns beyond the simple baseline.

- **Step 8: Compare Sample Predictions**

```

for i in range(10):
    print(
        f"Actual={y_test[i]:.3f}, "
        f"Baseline={baseline_pred[i]:.3f}, "
        f"RNN={rnn_pred[i]:.3f}"
    )

```

```

Actual=-0.152, Baseline=-0.119, RNN=-0.148
Actual=-0.165, Baseline=-0.152, RNN=-0.052
Actual=-0.099, Baseline=-0.165, RNN=-0.199
Actual=-0.279, Baseline=-0.099, RNN=0.059
Actual=0.028, Baseline=-0.279, RNN=-0.275
Actual=-0.443, Baseline=0.028, RNN=-0.336
Actual=-0.343, Baseline=-0.443, RNN=-0.180
Actual=-0.435, Baseline=-0.343, RNN=-0.330
Actual=-0.099, Baseline=-0.435, RNN=-0.138
Actual=-0.390, Baseline=-0.099, RNN=-0.434

```

We can also visualize the predictions.

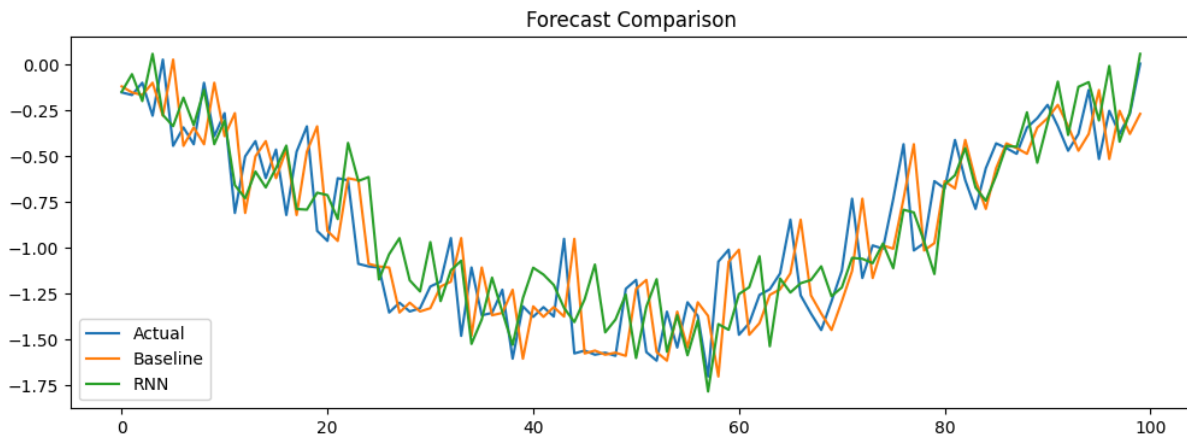
```

plt.figure(figsize=(12,4))

plt.plot(y_test[:100], label='Actual')
plt.plot(baseline_pred[:100], label='Baseline')
plt.plot(rnn_pred[:100], label='RNN')

plt.legend()
plt.title("Forecast Comparison")
plt.show()

```



The baseline tends to lag behind sudden changes because it simply repeats the previous value. The RNN can anticipate future movements by learning patterns from multiple previous observations.

- **Discussion**

At first glance, the baseline appears almost too simple to be useful. Yet it serves an important purpose. Suppose our results were: **Baseline MAE = 0.21; RNN MAE = 0.25**

Even though the RNN is far more sophisticated, it performs worse than the baseline. This tells us that the neural network has failed to learn useful information.

On the other hand, if the RNN consistently outperforms the baseline, then we have evidence that the additional complexity is justified.

This is why experienced practitioners almost always build a baseline first.

- Before asking: *"How good is my model?"*
- They ask: *"Is my model better than the simplest reasonable solution?"*

Summary

Baseline models are simple reference models used to evaluate whether a Machine Learning model is actually learning useful patterns.

In time-series forecasting, one of the most common baselines is the naive forecast, where the next value is assumed to be equal to the most recent

observed value. Although extremely simple, this baseline is often surprisingly difficult to beat.

By comparing the baseline against an RNN, we obtain a meaningful benchmark. If the RNN achieves lower prediction error, we can conclude that it has learned patterns beyond what the baseline captures. If it cannot outperform the baseline, then the additional complexity may not be worthwhile.

The key lesson is that Machine Learning performance is always relative. A model is not judged solely by its accuracy or loss, but by how much it improves upon a reasonable baseline.

References

<https://towardsdatascience.com/baseline-models-your-guide-for-model-building-1ec3aa244b8d/>

https://medium.com/@preethi_prakash/understanding-baseline-models-in-machine-learning-3ed94f03d645

Next Section:

 [S2. Training & Validation Metrics](#)